

Curso de Automatización con Java, Selenium, JUnit y Gherkin

Clase 11 – Cucumber + Selenium + Reporting

COLEGIO
UNIVERSITARIO



CCC
Centro de Capacitación
Continua

Temario

- ♦ Fundamentos de BDD (Behavior Driven Development)
- ♦ Lenguaje Gherkin y sintaxis básica
- ♦ Buenas prácticas y reglas de escritura en Gherkin

 **Práctica: Rescritura de escenarios imperativos a declarativos**

¿Qué es Cucumber?

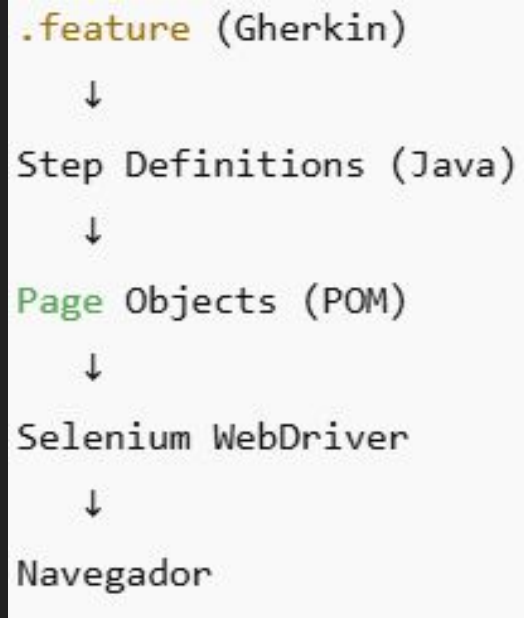
Cucumber es la herramienta que permite:

- Leer archivos .feature escritos en Gherkin
- Mapear cada paso a código Java
- Ejecutar escenarios como pruebas automatizadas

👉 Cucumber es el **punto** entre:

- BDD (lenguaje)
- Selenium (automatización)
- JUnit (ejecución)

Arquitectura general BDD + Selenium



Dependencias necesarias (Maven)

Agregar al pom.xml:

```
<!-- https://mvnrepository.com/artifact/io.cucumber/cucumber-java -->
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-java</artifactId>
  <version>7.33.0</version>
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/org.junit.platform/junit-platform-suite-api -->
<dependency>
  <groupId>org.junit.platform</groupId>
  <artifactId>junit-platform-suite-api</artifactId>
  <version>6.1.0-M1</version>
  <scope>test</scope>
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/org.junit.platform/junit-platform-suite-engine -->
<dependency>
  <groupId>org.junit.platform</groupId>
  <artifactId>junit-platform-suite-engine</artifactId>
  <version>6.1.0-M1</version>
  <scope>test</scope>
</dependency>
```

```
<!-- https://mvnrepository.com/artifact/io.cucumber/cucumber-junit-platform-engine -->
<dependency>
  <groupId>io.cucumber</groupId>
  <artifactId>cucumber-junit-platform-engine</artifactId>
  <version>7.33.0</version>
  <scope>test</scope>
</dependency>
```

Estructura del proyecto con Cucumber

```
src
├── test
│   ├── java
│   │   ├── runners
│   │   │   └── TestRunner.java
│   │   ├── stepdefinitions
│   │   │   └── LoginSteps.java
│   │   ├── base
│   │   │   └── BaseTest.java
│   │   ├── pages
│   │   │   └── LoginPage.java
│   │   └── utils
│   └── resources
│       └── features
│           └── login.feature
```

Feature (ejemplo)

login.feature

Feature: Login de usuario

Scenario: Login exitoso

Given el usuario se encuentra en la página de login

When ingresa usuario "standard_user" y contraseña "secret_sauce"

Then debería acceder a la página principal

Step Definitions

- Cada paso del feature se mapea a un método Java
- El texto debe coincidir exactamente (o con expresiones)

To illustrate how this works, look at the following Gherkin Scenario:

```
Scenario: Some cukes  
  Given I have 48 cukes in my belly
```

Gherkin scenario

The `I have 48 cukes in my belly` part of the step (the text following the `Given` keyword) will match the following step definition:

```
package com.example;  
import io.cucumber.java.en.Given;  
  
public class StepDefinitions {  
    @Given("I have {int} cukes in my belly")  
    public void i_have_n_cukes_in_my_belly(int cukes) {  
        System.out.format("Cukes: %n\n", cukes);  
    }  
}
```

Corresponding cucumber code

```

public class LoginSteps {

    LoginPage loginPage;
    HomePage homePage;

    @Given("el usuario se encuentra en la pagina de login")
    public void el_usuario_se_encuentra_en_la_pagina_de_login() {
        //Logica necesaria
        //1- Vamos a la pagina de login
        driver.get("https://www.saucedemo.com/");

        //2- Creamos las instancias de las pages que necesitaremos
        loginPage = new LoginPage(driver);
        homePage = new HomePage(driver);
    }

    @When("ingresa usuario {string} y contrasena {string}")
    public void ingresa_usuario_y_contrasena(String userName, String userPass) {
        loginPage.hacerLogin(userName, userPass);
    }

    @Then("deberia acceder a la pagina principal")
    public void deberia_acceder_a_la_pagina_principal() {
        Assertions.assertTrue(
            homePage.esVisibleElCarrito(), "El carrito no es visible");
    }
}

```

Agregamos los Hooks al comienzo de la clase de los steps, cuyo imports ahora son de cucumber

```
import io.cucumber.java.After;  
import io.cucumber.java.Before;
```

```
public class LoginSteps {  
.  
.  
.  
  
    protected WebDriver driver; //driver = null;  
  
    @Before  
    public void setup() {  
        //Configuracion del navegador  
        WebDriverManager.firefoxdriver().setup();  
  
        //Creacion del driver  
        driver = new FirefoxDriver();  
  
        //Indicamos que se maximize la ventana del navegador  
        driver.manage().window().maximize();  
    }  
  
    @After  
    public void tearDown() {  
        // Realizamos la validacion por si no se pudo ejecutar correctamente  
        //driver = new FirefoxDriver();  
        if (driver != null) {  
            //Cierra el navegador del finalizar la ejecucion del test  
            driver.quit();  
        }  
    }  
.  
.  
.  
}
```

Runner de Cucumber

El Runner conecta:

- Features
- Step Definitions
- Reportes

```
package runners;

import org.junit.platform.suite.api.ConfigurationParameter;
import org.junit.platform.suite.api.IncludeEngines;
import org.junit.platform.suite.api.SelectClasspathResource;
import org.junit.platform.suite.api.Suite;

import static io.cucumber.core.options.Constants.GLUE_PROPERTY_NAME;
import static io.cucumber.core.options.Constants.PLUGIN_PROPERTY_NAME;

@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
@ConfigurationParameter(
    key = GLUE_PROPERTY_NAME, value = "stepsdefinitions"
)
@ConfigurationParameter(
    key = PLUGIN_PROPERTY_NAME, value = "pretty, html:target/cucumber-report.html"
)
public class TestRunner {
}
```

Ejecución de pruebas

Desde IDE

- Click derecho sobre **TestRunner**
- Run

Desde Maven

```
mvn test
```

- ✓ Ejecuta todos los .feature
- ✓ Genera reportes automáticamente

Reportes de ejecución

Reporte HTML (Cucumber)

Ubicación:

`target/cucumber-report.html`

Contiene:

- Features ejecutadas
- Escenarios
- Pasos exitosos / fallidos
- Mensajes de error

Práctica

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

Ejercicio guiado

Automatizar los siguientes casos

Feature: Login de usuario

Background:

Given el usuario se encuentra en la página de login

Scenario: Login exitoso con credenciales válidas

When ingresa usuario "standard_user" y contraseña "secret_sauce"

Then debería acceder a la página principal

Scenario: Login fallido con usuario bloqueado

When ingresa usuario "locked_out_user" y contraseña "secret_sauce"

Then debería ver un mensaje de error "Epic sadface: Sorry, this user has been locked out."

Cierre

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

Buenas prácticas con Cucumber

- Features **100% declarativos**
- No poner lógica en los steps
- Reutilizar Page Objects
- Un step = una acción clara
- No duplicar steps innecesarios
- Mantener los steps genéricos
- Assertions solo en Then
- No usar sleeps

¿Preguntas?

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua

¡Gracias!

**COLEGIO
UNIVERSITARIO**



CCC
Centro de Capacitación
Continua